🚀 Lecture 02.03: Data Types & Type System Mechanics

📌 Executive Architectural Summary

A data type classifies the nature of a value, determines the operations that can be legally performed upon it, and dictates how many bytes are allocated in system memory. Python features a **dynamically and strongly typed** type system.

🔬 Core Engineering Mechanics

1. Primary Built-in Data Types

1. `int` **(Integer):** Arbitrary-precision whole numbers ($\mathbb{Z}$). No fixed 32-bit or 64-bit overflow limits in Python 3.
2. `float` **(Floating-Point):** Double-precision 64-bit IEEE-754 format (1 sign bit, 11 exponent bits, 52 mantissa bits).
3. `str` **(String):** Immutable sequence of Unicode code points.
4. `bool` **(Boolean):** Subclass of integer (`int`), possessing exactly two canonical singletons: `True` (1) and `False` (0).
5. `NoneType` **:** The singleton object `None` , representing the intentional absence of a value or null state.
6. `complex` **:** Complex numbers represented as $a + bj$ with real and imaginary parts stored as 64-bit floats.

2. Dynamic Typing vs Static Typing

- **Dynamic Typing:** Type checking occurs at runtime during execution rather than at compile-time.
- **Strong Typing:** Implicit illegal type conversions (e.g., `"5" + 5`) raise a `TypeError` rather than coercing silently.

💻 Production Python Implementation

```
# 1. Type Instantiations
val_int = 42
```

```
val_float = 3.14159
val_str = "Machine Learning"
val_bool = True
val_none = None
val_complex = 2 + 3j

# 2. Type Introspection
print(f"{val_int} -> {type(val_int)}")
print(f"{val_float} -> {type(val_float)}")
print(f"{val_str} -> {type(val_str)}")
print(f"{val_bool} -> {type(val_bool)} (isinstance int: {isinstance(val_bool, int)
print(f"{val_none} -> {type(val_none)}")
print(f"{val_complex} -> {type(val_complex)} (Real: {val_complex.real}, Imag: {val
```